Zenith: evaluating revisions to a reasoning graph

Will Cook • 12 October 2026

Experimental research prototype. The public example uses synthetic observations of a disclosed toy generator. No predictive improvement has been demonstrated.

Abstract. Zenith studies forecasting a changing, partially observed system through heterogeneous, incomplete and delayed observations. Lab freezes predictions; Oracle evaluates later evidence; Evolve proposes one revision for a held-out matched comparison. The synthetic fixture prepares the first request with zero model calls, no completed forecast and no demonstrated improvement.

1. Evaluation protocol. Reasoning reconciles meanings and clocks, distinguishes missing from conflicting evidence, proposes competing explanations and selects explicit forecast targets. The DAG records computational dependencies, not physical causality or independent votes. Keep the model, targets, score and budget fixed during comparison.

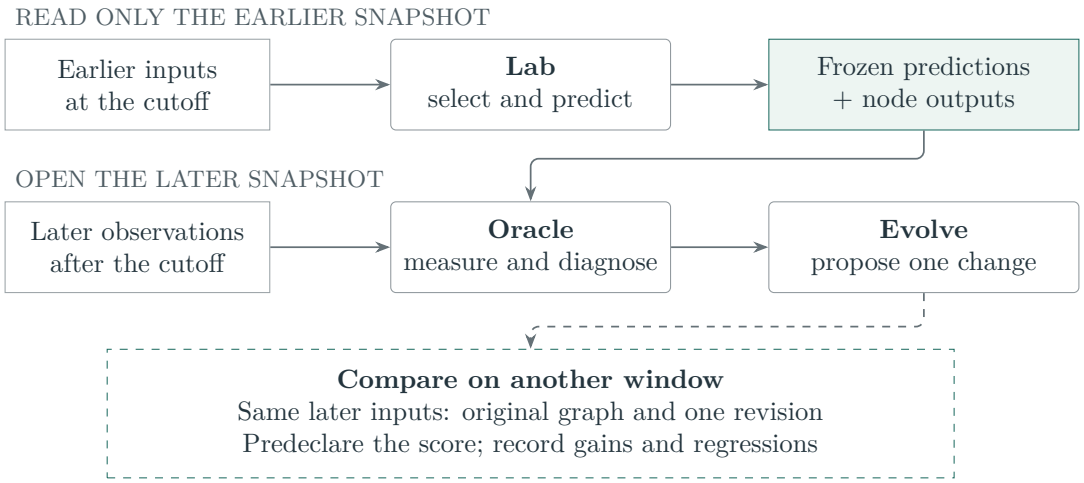

Figure 1. Proposed evaluation loop. Preserve the original record before revealing the outcome. The dashed step is a separate comparison, not evidence that a diagnosis or revision is correct.

Synthetic observations. Five source perspectives describe the same abstract System A: measurements, subsystems, environment, events and interventions. A sixth derives summaries from their supplied values. The disclosed generator uses dimensionless values, seed 17 and the prescribed recurrence $x_{n+1} = 0.7x_n + 0.2c_n + u_n + \epsilon_n$, with seeded noise. This is a toy construction, not a learned model or physically validated simulation. Provenance and receipts identify the inputs [2].

Physical motivation. Sensors expose different aspects of a changing state; history may recover information absent from one instant, but some state can remain unobservable [1]. Heat-transfer observations could support a six-hour temperature target; position, acceleration and force histories could support a one-second vibration target. Known forces, boundaries and interventions condition the forecast. Proposed reasoning aligns clocks, units and source dependence, compares sensor-fault and state-change explanations, and selects an appropriate physical model or numerical tool. Solver integration is unimplemented and untested; narrative cannot substitute for equations. System A remains a separate toy. Match entity, target, horizon and later truth.

Collect runs with the graph fixed; count unique cases across overlapping windows. Freeze one revision before opening comparison outcomes. Cases used to choose it become development data. The fixture does not establish that reasoning adds value over direct or numerical baselines.

Prior work. DSPy composes LM modules [6]; MIPRO optimises their instructions and examples against a metric [7]. AlphaGo Zero [3] and autoresearch [4] motivated successive comparisons. Their results do not establish a forecasting advantage here.

2. The executable graph and failure diagnosis. Six feed-specific miners analyse the permitted inputs. Saved outputs let a reviewer trace an unsupported claim to its first occurrence and actual evidence.

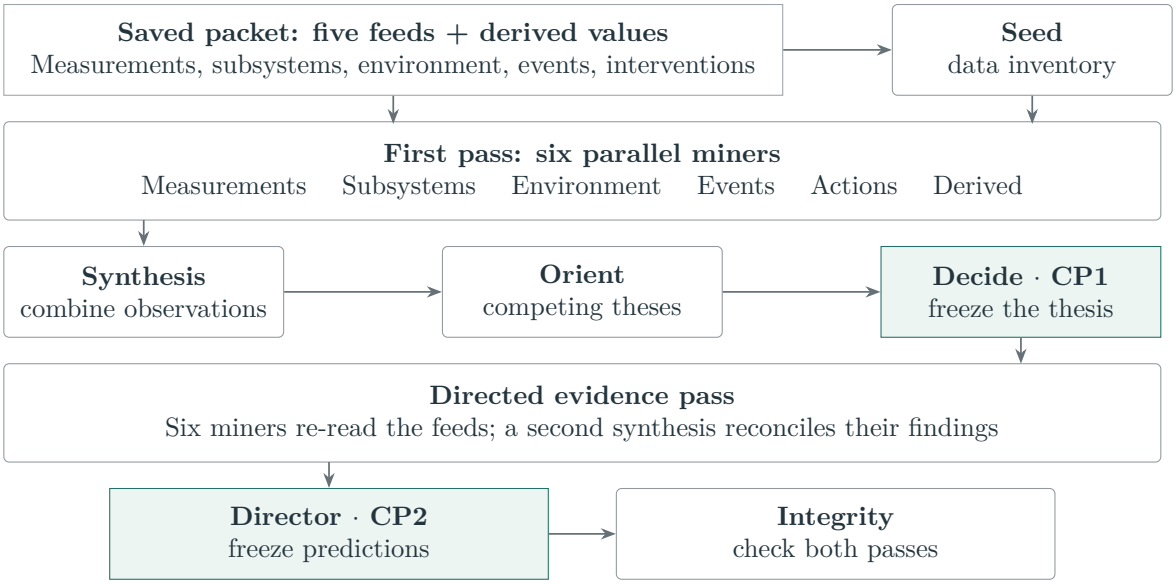

Figure 2. Lab's two passes. "Synthesis" denotes the nodes named Cross-correlation in the implementation; the second synthesis is grouped with the directed pass. CP1 is the saved thesis; CP2 freezes the declared target predictions.

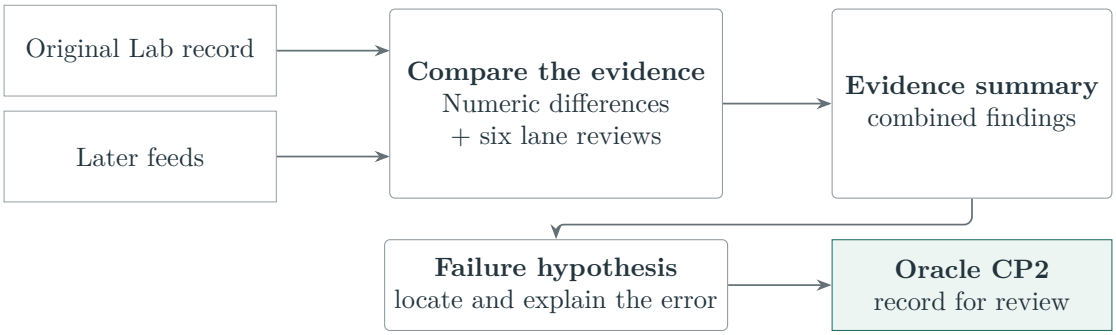

Figure 3. Oracle reviews the saved run against later evidence. Its attribution is a hypothesis about a failure, not a causal identification. Square boxes are records; rounded boxes are operations or groups. Exact feed, skip and cross-pass dependencies are retained in `docs/graphs.md`.

Diagnosis separates evidence absent from the packet, evidence omitted from a node's inputs, unsupported reasoning, incorrect prediction and execution failure. Keep unresolved cases unknown. Post-outcome research may explain an event but cannot become earlier evidence. Recorded interventions and repeated failures do not by themselves identify causes.

Evolve proposes a bounded instruction, context or dependency change. Compare original and revised graphs on the same further cases with the model and budget fixed. Report selection coverage, forecast error, failures and cost separately, with a score suited to the declared forecast [5]. A selected subset may look accurate while missing useful targets; declare how abstentions count.

3. Proposed context-conditioned prompts. A shared neural controller would use variability, missingness, disagreement and event density available at the cutoff to select small text variants for each node. Its outputs are *prompt settings*: numbers in $[0, 1]$ indexing a versioned catalogue, distinct from learned controller weights. The node's task, permitted inputs, output contract and work budget stay fixed. This extension has not been implemented or evaluated.

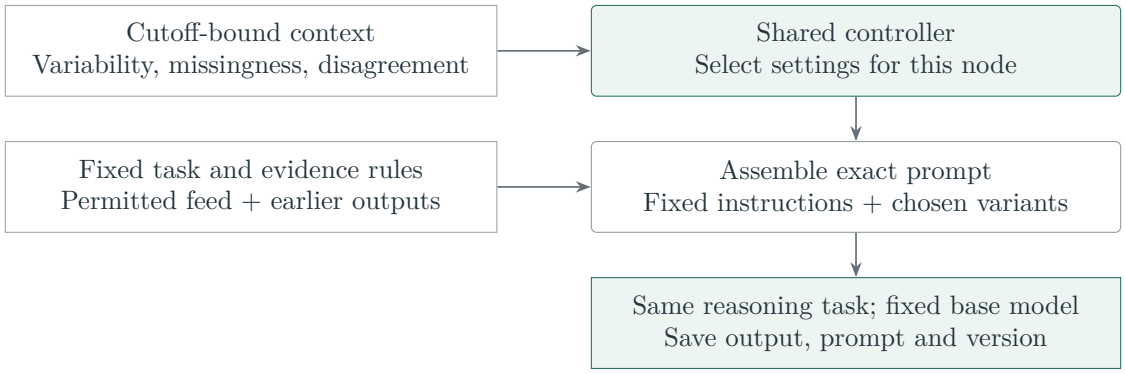

Figure 4. Prediction-time selection with a previously frozen controller. For the interventions miner, settings may vary caution about attribution while keeping seed state, interventions and evidence rules fixed. Temporal association alone is not causal proof. Settings select text, not numerical attention weights.

Development is separate: measure configurations on development cases, fit or select the controller, then freeze it before later testing. Search a small catalogue first, following discrete prompt optimisation [7]. Backpropagation could train the numerical controller from measured losses; hard text selection and hosted LM calls expose no ordinary end-to-end derivative. ProTeGi [8] and TextGrad [9] use verbal feedback as a gradient analogy. A critique suggests a revision; it is not a derivative or proof of benefit. Nearby settings need not behave smoothly or monotonically.

Verified boundary and next evaluation. The native graph loads the six synthetic artifacts and prepares the Seed request with zero model calls [2]. There is no completed CP1, CP2, forecast, Oracle diagnosis or Evolve comparison. Historical results belong to their original configurations. Complete Lab, score the frozen predictions against later observations and compare one revision on further matched cases. Test any controller against fixed prompts under the same information and budget, including regressions and uncertainty. Synthetic preparation establishes neither real-world predictive performance nor universal reliability. Detailed protocol and controller choices remain in `docs/research-direction.md`.

References

[1] R. Tedrake. *Underactuated Robotics*, Output Feedback. underactuated.mit.edu/output_feedback.html.

[2] Zenith. Synthetic observation demonstration, per-feed provenance and preparation receipt, 2026.

[3] D. Silver et al. Mastering the game of Go without human knowledge. *Nature* 550:354–359, 2017. doi:10.1038/nature24270.

[4] A. Karpathy. *autoresearch*. Source inspected at `228791fb499a` (code).

[5] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *JASA* 102(477):359–378, 2007. doi:10.1198/016214506000001437.

[6] O. Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *ICLR*, 2024. arXiv:2310.03714.

[7] K. Opsahl-Ong et al. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. *EMNLP*, 2024. arXiv:2406.11695v2.

[8] R. Pryzant et al. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. *EMNLP*, 2023. arXiv:2305.03495.

[9] M. Yuksekgonul et al. TextGrad: Automatic "Differentiation" via Text. 2024. arXiv:2406.07496v1.